

HOGWARTS LEGACY

HOGWARTS LEGACY 5.0 | 24-HOUR HACKATHON

ARTIFICIAL INTELLIGENCE DOMAIN FUNCTIONALITY GUIDE

Development-ready feature breakdown for all 10 problem statements

Prepared for participants and organizers

How to Use This Guide

Each problem statement includes the original scenario, an objective summary, recommended core functionalities, and an expected demonstration flow. For a 24-hour event, teams should prioritize a working prototype of the core workflow and use sample or simulated data where appropriate.

01. AI-Based UAT Test Case Generator

Scenario

Software teams often spend significant time manually converting business workflows and requirements into detailed User Acceptance Testing (UAT) test cases.

Objective

Convert business requirements and workflows into structured User Acceptance Testing test cases.

Functionalities to Implement

- 1. Requirement Input:** Accept text, user stories, workflow descriptions, or uploaded documents.
- 2. Requirement Understanding:** Extract business rules, roles, actions, outcomes, conditions, and dependencies.
- 3. Scenario Generation:** Create positive, negative, and boundary test scenarios.
- 4. Test Case Generation:** Include test ID, scenario, preconditions, steps, test data, and expected result.
- 5. Role-Based Cases:** Generate cases for different roles and permission levels.
- 6. Validation:** Flag duplicates, incomplete cases, and requirements lacking detail.
- 7. Review & Editing:** Allow users to review, edit, and regenerate cases.
- 8. Export & Dashboard:** Display cases by scenario, priority, and role; export to Excel or CSV.

Expected Demo: Upload a workflow → generate scenarios and test cases → review/edit → export the test suite.

02. AI-Powered Lead Qualification & CRM Bot

Scenario

Businesses receive enquiries from multiple customers, but manually collecting information and identifying high-potential leads can be time-consuming.

Objective

Collect customer enquiries, evaluate lead quality, and manage leads through a CRM workflow.

Functionalities to Implement

- 1. Customer Chatbot:** Provide a chat interface and understand natural-language enquiries.
- 2. Information Collection:** Collect name, contact details, requirements, budget, and timeline; ask follow-up questions when needed.
- 3. Information Extraction:** Convert conversation details into structured fields.
- 4. Qualification Engine:** Evaluate leads using configurable budget, need, timeline, and business-fit criteria.
- 5. Lead Scoring:** Assign scores and categorize leads as High, Medium, or Low priority.
- 6. Lead Categorization:** Classify leads by product/service interest and enquiry type.

7. CRM Management: Create, edit, and store leads in a database.

8. Assignment & Pipeline: Assign leads to representatives and track New, Qualified, Contacted, and Converted stages.

9. Dashboard & Summary: Show leads, scores, statuses, conversion statistics, and concise conversation summaries.

Expected Demo: Customer chats → bot collects details → qualifies and scores lead → saves it to the CRM dashboard.

03. Mission Failure & Recovery Simulator

Scenario

Autonomous drones may encounter unexpected situations such as GPS loss, low battery, communication failure, or obstacles during a mission.

Objective

Simulate drone mission failures and demonstrate how an autonomous system detects and responds.

Functionalities to Implement

1. Mission Simulator: Create a visual environment with drone, start point, and destination.

2. Mission Configuration: Set mission parameters and destination.

3. Failure Scenarios: Simulate GPS loss, low battery, communication failure, and obstacles.

4. Failure Detection: Monitor simulated status and detect predefined failure conditions.

5. Failure Classification: Identify failure type, severity, and potential mission impact.

6. Recovery Decision Engine: Select actions such as return-to-home, hover, or alternate route based on failure.

7. Recovery Simulation: Execute the action in simulation and show whether recovery succeeds.

8. Dashboard & Timeline: Display battery, GPS, communication, mission state, failures, recovery decisions, and final outcome.

Expected Demo: Start a simulated mission → trigger a failure → detect it → select recovery → display the mission outcome.

04. Computer Vision-Based Defect Detection

Scenario

Manufacturing industries need to identify defective products quickly and consistently during quality inspection.

Objective

Identify and classify predefined defects in manufactured product images or camera input.

Functionalities to Implement

1. Image Input: Upload product images or use live camera input where available.

2. Preprocessing: Resize and normalize images for model input.

3. Defect Detection: Use a trained computer-vision model to identify predefined defects.

4. Defect Classification: Classify scratches, cracks, dents, or other selected defect categories.

5. Defect Localization: Highlight defects with bounding boxes or segmentation masks, depending on the model.

6. Confidence Scoring: Display model confidence for each detection.

7. Acceptance Decision: Mark products Accepted or Rejected using predefined rules.

8. Inspection History & Dashboard: Store results and show defect categories and acceptance/rejection statistics.

9. Inspection Report: Include image, detected defects, and final decision.

Expected Demo: Upload a product image → detect and highlight defects → classify them → show acceptance/rejection.

05. Rover Path Planning in Simulated Terrain

Scenario

Autonomous rovers operating in unknown or challenging environments need to reach specific destinations while avoiding obstacles.

Objective

Plan a safe and efficient route for a rover through an obstacle-filled simulated environment.

Functionalities to Implement

1. Terrain Simulator: Create a grid/map with free paths, obstacles, and restricted areas.

2. Start/Destination: Let users select starting point and destination.

3. Obstacle Configuration: Add/remove obstacles and choose preset terrain layouts.

4. Pathfinding: Implement A*, Dijkstra, or another suitable algorithm to find a safe route.

5. Path Optimization: Minimize distance or movement cost using defined criteria.

6. Movement Simulation: Animate rover movement and display current position.

7. Dynamic Obstacles: Introduce obstacles during movement and recalculate the route.

8. Visualization & Metrics: Show planned/actual path, distance, steps, and planning time.

9. Mission Summary: Report whether the rover reached its destination and the route taken.

Expected Demo: Create terrain → set start and destination → plan and animate route → add an obstacle → show replanning.

06. IoT-Based Equipment Anomaly Detection

Scenario

Industrial equipment can develop faults when parameters such as temperature, vibration, or power consumption move outside normal operating patterns.

Objective

Monitor equipment sensor data and identify abnormal conditions early.

Functionalities to Implement

1. Equipment Registration: Register equipment with unique IDs and types.

2. Sensor Data: Collect or simulate temperature, vibration, and power readings.

3. Live Monitoring: Display incoming readings and update equipment status.

4. Normal Baseline: Define normal ranges or expected patterns for each sensor.

5. Anomaly Detection: Detect abnormal values or patterns using thresholds or a model.

- 6. Anomaly Classification:** Categorize overheating, excessive vibration, or unusual power consumption.
- 7. Health Scoring:** Calculate equipment health/risk from sensor readings.
- 8. Alerts & Dashboard:** Trigger alerts and show readings, health, anomaly history, and affected equipment.
- 9. Monitoring Report:** Summarize anomalies and recommended inspection actions.

Expected Demo: Simulate sensor data → introduce abnormal readings → detect anomaly → alert and display equipment health.

07. AI-Powered College Helpdesk Assistant

Scenario

Students frequently need information about examinations, academic processes, departments, events, and various college services.

Objective

Answer student questions using an approved college knowledge base.

Functionalities to Implement

- 1. Chat Interface:** Allow students to ask natural-language questions.
- 2. Knowledge Base:** Upload academic calendars, exam schedules, regulations, and department information; organize by category.
- 3. Document Processing:** Extract text and split documents into searchable sections.
- 4. Knowledge Retrieval:** Retrieve relevant information through semantic or keyword search.
- 5. AI Response:** Generate answers grounded in retrieved information.
- 6. Source Citations:** Show the supporting document or section for each answer.
- 7. Query Categories:** Handle examinations, admissions, departments, events, and academic processes.
- 8. Fallback Handling:** State when information is unavailable and record unanswered queries.
- 9. Admin & Dashboard:** Let authorized staff update documents; show query history, FAQs, and unanswered questions.

Expected Demo: Upload college documents → ask a student question → retrieve information → answer with a source.

08. Tamper-Proof Academic Credential Verification

Scenario

Academic certificates can be difficult to verify manually, creating challenges for institutions, employers, and students.

Objective

Enable institutions to issue digital credentials and allow authorized users to verify authenticity.

Functionalities to Implement

- 1. Institution Registration:** Register credential-issuing institutions and maintain their status.
- 2. Credential Issuance:** Create certificates with student, qualification, institution, and issue-date details.
- 3. Unique Credential ID:** Generate a unique identifier for each credential.
- 4. QR Code:** Generate a QR code linking to the verification page.

5. **Integrity Protection:** Hash credential data and store the integrity reference securely.
6. **Credential Verification:** Verify using credential ID or QR code.
7. **Tampering Detection:** Detect modified, invalid, revoked, or unrecognized credentials.
8. **Revocation:** Allow authorized institutions to revoke credentials and show current status.
9. **Dashboard & History:** Display verification results and record issuance, revocation, and verification events.

Expected Demo: Issue a certificate → generate QR → verify it → alter a test record → demonstrate tampering detection.

09. Quantum vs Classical Optimization

Scenario

Real-world problems such as scheduling, routing, and resource allocation often involve finding the most efficient solution among many possible combinations.

Objective

Solve a selected optimization problem with classical and quantum or quantum-inspired approaches and compare results.

Functionalities to Implement

1. **Problem Selection:** Choose scheduling, routing, or resource allocation; define objective and constraints.
2. **Input Interface:** Configure parameters and provide sample datasets.
3. **Classical Engine:** Implement a suitable classical algorithm and produce a solution.
4. **Quantum-Based Engine:** Implement a quantum or quantum-inspired method using a simulator/framework if needed.
5. **Constraint Validation:** Check feasibility and identify violated constraints.
6. **Objective Calculation:** Calculate the objective value for each solution.
7. **Performance Comparison:** Compare solution quality, execution time, and feasibility.
8. **Visualization & Experiments:** Show results in tables/charts and allow parameter changes and repeated runs.
9. **Comparison Report:** Summarize settings, solutions, and metrics.

Expected Demo: Configure a problem → run both approaches → compare solutions, feasibility, and performance.

10. AI-Based Digital Habit Awareness System

Scenario

People spend significant time using digital devices and applications, but may not always understand their own usage patterns and habits.

Objective

Analyze digital usage patterns and provide personalized awareness insights.

Functionalities to Implement

1. **Usage Data Input:** Import or manually enter screen-time and application usage data.
2. **Application Tracking:** Track time by app and categorize usage.
3. **Pattern Analysis:** Identify daily/weekly trends, peak periods, and frequently used apps.
4. **Notification Analysis:** Measure notification frequency and high-notification periods.

5. Habit Detection: Identify long sessions, repeated patterns, and changes over time.

6. AI Insights: Generate personalized, explainable insights from usage data.

7. Recommendations: Suggest screen-time goals or reducing unnecessary notifications; let users configure goals.

8. Goal Tracking: Set daily targets and monitor progress.

9. Dashboard & Report: Show screen time, app usage, notifications, trends, goals, and recommendations.

Expected Demo: Import usage data → analyze habits → generate insights → show recommendations and goal progress.

Common Expectations for All Teams

- **Functional interface:** Provide a usable interface for entering data and viewing results.
- **Input validation:** Validate inputs and handle missing or invalid data.
- **Core workflow:** Implement the primary detection, analysis, generation, or simulation feature.
- **Results display:** Present results clearly using tables, cards, charts, or visualizations.
- **History where relevant:** Keep a record of previous operations or results when useful.
- **Error handling:** Show meaningful errors and handle failures gracefully.
- **Demo-ready sample data:** Include sample inputs so the prototype can be demonstrated reliably.
- **Reports where relevant:** Export results when appropriate to the selected problem.
- **End-to-end demonstration:** Show a complete flow from input through processing to output.